

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.
2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.
3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.
4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.
5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

RUBRICS FOR CALCULATION:

Q. No	Understanding of Concepts (25)	Experimenta l Design (30)	Use of Tools (20)	Analysis of Results (15)	Documenta tion (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

R. Deepasri (192511309)

CO3 AT1-Cryptography and NS(Experiments):

1. MD5 and SHA-256 Hashing

CODE:

```
import hashlib
import time
```

```
n = int(input("Enter number of messages: "))
```

```
for i in range(n):
```

```
    msg = input("Enter message: ")
```

```
    start = time.time()
```

```
    md5 = hashlib.md5(msg.encode()).hexdigest()
```

```
    md5_time = time.time() - start
```

```
    start = time.time()
```

```
    sha = hashlib.sha256(msg.encode()).hexdigest()
```

```
    sha_time = time.time() - start
```

```
    print("\nMessage:", msg)
```

```
    print("MD5  :", md5)
```

```
    print("SHA-256:", sha)
```

```
    print("MD5 Time:", md5_time)
```

```
    print("SHA-256 Time:", sha_time)
```

```
print("\nSHA-256 has stronger collision resistance than MD5.")
```

The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is as follows:

```

1 import hashlib
2 import time
3
4 n = int(input("Enter number of messages: "))
5
6 for i in range(n):
7     msg = input("Enter message: ")
8
9     start = time.time()
10    md5 = hashlib.md5(msg.encode()).hexdigest()
11    md5_time = time.time() - start
12
13    start = time.time()
14    sha = hashlib.sha256(msg.encode()).hexdigest()
15    sha_time = time.time() - start
16
17    print("\nMessage:", msg)
18    print("MD5      :", md5)
19    print("SHA-256:", sha)
20    print("MD5 Time:", md5_time)
21    print("SHA-256 Time:", sha_time)
22

```

The output of the program is:

```

Enter number of messages: 2
Enter message: hello

Message: hello
MD5       : 5d41402abc4b2a76b9719d911017c592
SHA-256: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e7304336293
8b9824
MD5 Time: 5.9604644775390625e-05
SHA-256 Time: 1.7404556274414062e-05
Enter message: hello world

Message: hello world
MD5       : 5eb63bbbe01eeed093cb22bb8f5acdc3
SHA-256: b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2
efcde9
MD5 Time: 2.5272369384765625e-05
SHA-256 Time: 5.4836273193359375e-06

SHA-256 has stronger collision resistance than MD5.

=== Code Execution Successful ===

```

2. RSA Digital Signature

```
import hashlib
```

```
p = 61
```

```
q = 53
```

```
n = p * q
```

```
phi = (p - 1) * (q - 1)
```

```
e = 17
```

```
d = pow(e, -1, phi)
```

```
msg = input("Enter message: ")
```

```
h = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % n
```

```
signature = pow(h, d, n)
```

```
print("\nHash:", h)
```

```
print("Signature:", signature)
```

```
received = input("Enter message for verification: ")
```

```
h2 = int(hashlib.sha256(received.encode()).hexdigest(), 16) % n
verify = pow(signature, e, n)
```

```
if h2 == verify:
    print("Signature Verified")
else:
    print("Signature Verification Failed")
```

The screenshot shows a Python Online Compiler interface. The code in the editor is as follows:

```
1 import hashlib
2
3 p = 61
4 q = 53
5 n = p * q
6 phi = (p - 1) * (q - 1)
7
8 e = 17
9 d = pow(e, -1, phi)
10
11 msg = input("Enter message: ")
12
13 h = int(hashlib.sha256(msg.encode()).hexdigest(), 16) % n
14
15 signature = pow(h, d, n)
16
17 print("\nHash:", h)
18 print("Signature:", signature)
19
20 received = input("Enter message for verification: ")
21
22 h2 = int(hashlib.sha256(received.encode()).hexdigest(), 16) %
```

The output window shows the following results:

```
Enter message: hello
Hash: 1796
Signature: 1444
Enter message for verification: hello
Signature Verified

=== Code Execution Successful ===
```

3. HMAC

```
import hashlib
import hmac
```

```
message = input("Enter message: ")
key = input("Enter secret key: ")
```

```
simple_hash = hashlib.sha256(message.encode()).hexdigest()
```

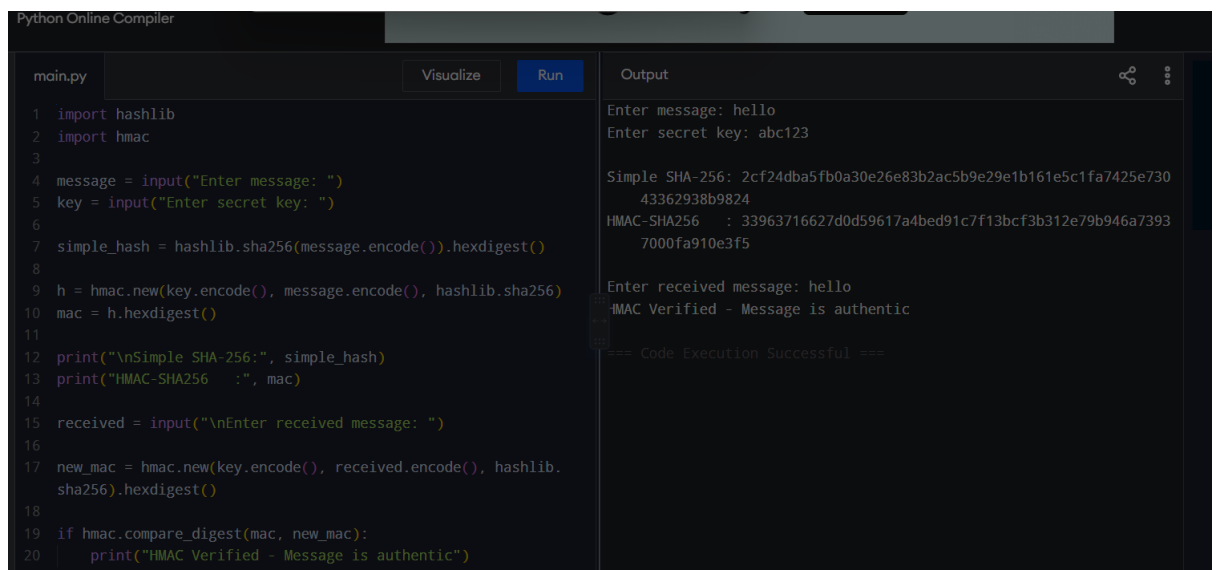
```
h = hmac.new(key.encode(), message.encode(), hashlib.sha256)
mac = h.hexdigest()
```

```
print("\nSimple SHA-256:", simple_hash)
print("HMAC-SHA256 :", mac)
```

```
received = input("\nEnter received message: ")
```

```
new_mac = hmac.new(key.encode(), received.encode(),  
hashlib.sha256).hexdigest()
```

```
if hmac.compare_digest(mac, new_mac):  
    print("HMAC Verified - Message is authentic")  
else:  
    print("HMAC Failed - Message was modified")
```



The screenshot shows a Python Online Compiler interface. On the left, a code editor displays a Python script for HMAC verification. The script imports hashlib and hmac, takes user input for a message and a secret key, calculates a simple SHA-256 hash and an HMAC, and then compares the HMAC of a received message to the original HMAC. On the right, the 'Output' pane shows the execution results, including the user inputs, the calculated hashes, and the final verification message: 'HMAC Verified - Message is authentic'. The code execution is marked as successful.

```
main.py Visualize Run Output
```

```
1 import hashlib
2 import hmac
3
4 message = input("Enter message: ")
5 key = input("Enter secret key: ")
6
7 simple_hash = hashlib.sha256(message.encode()).hexdigest()
8
9 h = hmac.new(key.encode(), message.encode(), hashlib.sha256)
10 mac = h.hexdigest()
11
12 print("\nSimple SHA-256:", simple_hash)
13 print("HMAC-SHA256   :", mac)
14
15 received = input("\nEnter received message: ")
16
17 new_mac = hmac.new(key.encode(), received.encode(), hashlib.
    sha256).hexdigest()
18
19 if hmac.compare_digest(mac, new_mac):
20     print("HMAC Verified - Message is authentic")
```

```
Enter message: hello
Enter secret key: abc123

Simple SHA-256: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e730
43362938b9824
HMAC-SHA256   : 33963716627d0d59617a4bed91c7f13bcf3b312e79b946a7393
7000fa910e3f5

Enter received message: hello
HMAC Verified - Message is authentic

=== Code Execution Successful ===
```

4.Simplified Kerberos Authentication

```
import time
```

```
import hashlib
```

```
password = input("Enter client password: ")
```

```
client_key = hashlib.sha256(password.encode()).hexdigest()
```

```
ticket = hashlib.sha256(  
    ("CLIENT" + client_key).encode()  
)hexdigest()
```

```

timestamp = time.time()

print("\nTicket generated:", ticket[:20] + "...")
print("Ticket sent to server.")

server_time = time.time()

if server_time - timestamp <= 10:
    print("Ticket Valid")
    print("Authentication Successful")
else:
    print("Ticket Expired")
    print("Authentication Failed")

```

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for ticket authentication. The output panel on the right shows the results of running the code, including the input password, the generated ticket, and the successful authentication message.

```

main.py Visualize Run
1 import time
2 import hashlib
3
4 password = input("Enter client password: ")
5
6 client_key = hashlib.sha256(password.encode()).hexdigest()
7
8 ticket = hashlib.sha256(
9     ("CLIENT" + client_key).encode()
10 ).hexdigest()
11
12 timestamp = time.time()
13
14 print("\nTicket generated:", ticket[:20] + "...")
15 print("Ticket sent to server.")
16
17 server_time = time.time()
18
19 if server_time - timestamp <= 10:
20     print("Ticket Valid")
21     print("Authentication Successful")

```

Output

```

Enter client password: abc123

Ticket generated: 3882fe260da60eb9fa92...
Ticket sent to server.
Ticket Valid
Authentication Successful

=== Code Execution Successful ===

```

5.ElGamal Digital Signature

```
import hashlib
```

```
p = 467
```

```
g = 2
```

```
x = 127
```

```
y = pow(g, x, p)
```



```
message = input("Enter message: ")

m = int(hashlib.sha256(message.encode()).hexdigest(), 16) % (p - 1)

k = 3

while __import__('math').gcd(k, p - 1) != 1:
    k += 1

r = pow(g, k, p)
k_inv = pow(k, -1, p - 1)

s = ((m - x * r) * k_inv) % (p - 1)

print("\nPublic Key:", (p, g, y))
print("Signature:", (r, s))

v1 = pow(y, r, p) * pow(r, s, p) % p
v2 = pow(g, m, p)

if v1 == v2:
    print("Signature Verified")
else:
    print("Signature Verification Failed")
```

main.py

Visualize

Run

Output

```
1 import hashlib
2
3 p = 467
4 g = 2
5 x = 127
6
7 y = pow(g, x, p)
8
9 message = input("Enter message: ")
10
11 m = int(hashlib.sha256(message.encode()).hexdigest(), 16) %
    (p - 1)
12
13 k = 3
14
15 while __import__('math').gcd(k, p - 1) != 1:
16     k += 1
17
18 r = pow(g, k, p)
19 k_inv = pow(k, -1, p - 1)
20
21 s = ((m - x * r) * k_inv) % (p - 1)
```

Enter message: hello

Public Key: (467, 2, 132)

Signature: (8, 202)

Signature Verified

=== Code Execution Successful ===